

ORIENTADOR.IA

Especificação Técnica para Agentes de Desenvolvimento Autônomos

Stack Principal	Google ADK + Ollama + FastAPI + Next.js
Gestão de Pacotes Python	UV (obrigatório, substituir pip)
LLMs	Ollama (modelos locais open-source)
Scope	MVP — Fase 1 (Foundation & Static Agents)
Versão	1.0 — Março 2026

Este documento é o prompt único e autoritário para construção do MVP. Cada instrução é obrigatória. Não há espaço para interpretação — o que não está aqui especificado, não deve ser implementado no MVP.

0. CONTEXTO E REGRAS ABSOLUTAS DO AGENTE

Este documento define de forma completa e sem ambiguidade o MVP do sistema Orientador.IA. O agente de desenvolvimento deve ler este documento na íntegra antes de executar qualquer comando ou escrever qualquer linha de código.

0.1 Identidade do Sistema

Orientador.IA é uma plataforma de orientação académica baseada em Sistemas Multi-Agente. O sistema utiliza avatares teóricos e metodológicos ('Almas') que dialogam com estudantes via maiêutica socrática, constroem um Canvas de Investigação de forma colaborativa e fornecem curadoria bibliográfica sem alucinações via RAG.

0.2 Regras Absolutas de Implementação

⚠ **OBRIGATÓRIO:** Todo o código Python DEVE usar UV como gestor de pacotes. NUNCA usar pip diretamente.

⚠ **OBRIGATÓRIO:** Todos os LLMs DEVEM ser servidos via Ollama. NUNCA fazer chamadas à API OpenAI, Anthropic ou Google Gemini.

⚠ **OBRIGATÓRIO:** A orquestração multi-agente DEVE usar exclusivamente Google Agent Development Kit (ADK). NUNCA implementar LangGraph, AutoGen ou LangChain.

⚠ **OBRIGATÓRIO:** O MVP cobre exclusivamente os módulos definidos na Secção 3. Qualquer funcionalidade fora do MVP está proibida nesta iteração.

⚠ **OBRIGATÓRIO:** Toda a comunicação entre agentes usa o GraphState definido na Secção 4.4. Nenhum estado fora deste schema é permitido.

📌 **NOTA:** Quando um requisito disser DEVE, é mandatório. Quando disser PODE, é opcional no MVP mas o código deve ser estruturado para suportá-lo no futuro.

1. ARQUITECTURA DO MVP

1.1 Diagrama Lógico de Componentes

O MVP é composto pelos seguintes serviços, todos orquestrados via Docker Compose:

Serviço	Tecnologia	Porto	Responsabilidade
frontend	Next.js 14 + TypeScript	3000	Interface do utilizador (Chat, Canvas, Projetos)
backend	FastAPI (Python 3.12)	8000	API REST + WebSocket + Orquestração ADK
ollama	Ollama Server	11434	Serve os LLMs localmente (Llama 3, Mistral, Nomic)
postgres	PostgreSQL 16	5432	Dados relacionais (utilizadores, projetos, canvas)
qdrant	Qdrant v1.9	6333 / 6334	Base de dados vetorial (RAG, embeddings)
redis	Redis 7	6379	Cache de sessões e fila de tarefas async

1.2 Fluxo de Dados Principal (Chat Request)

A sequência obrigatória para cada mensagem de chat é:

- Utilizador envia mensagem → Frontend (WebSocket) → Backend /api/chat
- Backend valida JWT → carrega GraphState do Redis → injeta no ADK Runner
- ADK Orchestrator Agent analisa GraphState e decide: responder diretamente OU acionar sub-agente
- Alma seleccionada gera resposta via Ollama (streaming SSE para o Frontend)
- Canvas Extractor Agent analisa conversa em background → atualiza postgres + emite evento WebSocket
- Frontend recebe stream de texto + evento de atualização do Canvas

2. ESTRUTURA DE DIRECTÓRIOS OBRIGATÓRIA

O agente DEVE criar exactamente esta estrutura. Nenhum ficheiro fora desta árvore é necessário para o MVP.

```
orientador-ia/
├── .env # Variáveis de ambiente (ver Secção 2.1)
├── docker-compose.yml # Orquestração de todos os serviços
├── backend/
│   ├── pyproject.toml # UV project file (NUNCA requirements.txt)
│   ├── uv.lock # Lock file gerido pelo UV
│   ├── Dockerfile
│   └── app/
│       ├── main.py # FastAPI entrypoint
│       ├── config.py # Settings via pydantic-settings
│       ├── database.py # SQLAlchemy async engine
│       ├── models/
│       │   ├── sql_models.py # ORM models (User, Project, Canvas, Resource)
│       │   └── schemas.py # Pydantic schemas para API
│       ├── api/
│       │   ├── auth.py # Endpoints: /api/auth/*
│       │   ├── projects.py # Endpoints: /api/projects/*
│       │   └── chat.py # Endpoints: /api/chat (WebSocket + SSE)
│       ├── agents/
│       │   ├── orchestrator.py # ADK OrchestratorAgent ('O Maestro')
│       │   └── almas/
│       │       ├── base_alma.py # Classe base para todas as Almas
│       │       ├── foucault.py # Alma Teórica: Michel Foucault
│       │       ├── bourdieu.py # Alma Teórica: Pierre Bourdieu
│       │       ├── freire.py # Alma Teórica: Paulo Freire
│       │       ├── habermas.py # Alma Teórica: Jürgen Habermas
│       │       ├── hooks.py # Alma Teórica: bell hooks
│       │       ├── etnografa.py # Avatar Metodológico: A Etnógrafa Digital
│       │       ├── estatistico.py # Avatar Metodológico: O Estatístico
│       │       ├── fenomenologa.py # Avatar Metodológico: A Fenomenóloga
│       │       ├── grounded.py # Avatar Metodológico: O Grounded Theorist
│       │       └── narrativo.py # Avatar Metodológico: O Analista Narrativo
│       ├── canvas_extractor.py # ADK Agent: extrai Canvas do diálogo
│       ├── match_engine.py # ADK Agent: faz match de Almas
│       └── guardrails.py # ADK Agent: valida integridade académica
│   └── services/
│       ├── ollama_client.py # Wrapper para Ollama API
│       ├── qdrant_client.py # Wrapper para Qdrant
│       └── embedding.py # Geração de embeddings via Ollama/nomic-embed-text
│   └── state/
│       └── graph_state.py # GraphState Pydantic model (schema canónico)
├── frontend/
│   ├── package.json
│   ├── next.config.ts
│   ├── tailwind.config.ts
│   └── src/
│       ├── app/ # Next.js App Router
│       │   ├── (auth)/ # Route group: login, register
│       │   ├── (dashboard)/ # Route group: projects list
│       │   └── project/[id]/ # Route: atelier + canvas
```

```

|   |   | components/
|   |   | | chat/           # ChatWindow, MessageBubble, AlmaAvatar
|   |   | | canvas/        # CanvasPanel, CanvasField, CanvasExport
|   |   | | ui/             # Componentes shadcn/ui
|   |   | |
|   |   | | lib/
|   |   | | | api.ts         # fetch wrapper para o Backend
|   |   | | | ws.ts         # WebSocket client
|   |   | | |
|   |   | | | store/
|   |   | | | | project.ts   # Zustand store (canvas + chat state)
|   |   | | |
|   |   | | scripts/
|   |   | | | seed_almas.py  # Script para popular a tabela ecosystem_resources

```

2.1 Ficheiro .env (Variáveis Obrigatórias)

Criar na raiz do projeto. Todas as variáveis são obrigatórias para o MVP.

```

# — Base de Dados —
DATABASE_URL=postgresql+asyncpg://orientador:orientador_pass@postgres:5432/orientador_db

# — Redis —
REDIS_URL=redis://redis:6379/0

# — Ollama —
OLLAMA_BASE_URL=http://ollama:11434
OLLAMA_CHAT_MODEL=llama3.1:8b
OLLAMA_GUARDRAIL_MODEL=mistral:7b
OLLAMA_EMBED_MODEL=nomic-embed-text
OLLAMA_EMBED_DIMENSIONS=768

# — Qdrant —
QDRANT_HOST=qdrant
QDRANT_PORT=6333
QDRANT_COLLECTION=knowledge_base

# — Auth (JWT) —
SECRET_KEY=SUBSTITUA_POR_VALOR_ALEATORIO_DE_64_CARACTERES_HEX
ACCESS_TOKEN_EXPIRE_MINUTES=60

# — Frontend —
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000

```

3. SCOPE DO MVP — O QUE IMPLEMENTAR

O MVP cobre exatamente estes módulos. Qualquer outra funcionalidade do PRD original está fora do scope desta iteração.

Módulo	Funcionalidade	Incluído no MVP
Autenticação	Registo com email/password + login JWT	SIM
Projetos	CRUD de projetos (criar, listar, editar, apagar)	SIM
Projetos	Definir área, nível académico, título provisório	SIM
Match Engine	Sugerir 3 Almas Teóricas + 3 Avatares Metodológicos por similaridade semântica	SIM
Ateliê/Chat	Chat multi-agente com streaming de respostas (SSE)	SIM
Canvas	Extração automática do Canvas a partir do diálogo	SIM
Canvas	Edição manual dos campos do Canvas	SIM
Canvas	Exportação do Canvas como JSON	SIM (PDF/DOCX: fora do MVP)
Guardrails	Deteção de pedidos de plágio e bloqueio	SIM
RAG	Pipeline RAG completo com Agente Buscador externo	NÃO (Fase 2)
Mesa-Redonda	Análise de dados empíricos (upload de transcrições)	NÃO (Fase 3)
Agente Ferramenteiro	Geração dinâmica de scripts em Sandbox	NÃO (Fase 4)
Agente Génesis	Criação dinâmica de novas Almas	NÃO (Fase 4)
OAuth	Login com Google / Institucional	NÃO (Fase 2)

4. BACKEND — ESPECIFICAÇÃO DETALHADA

4.1 Configuração do Projecto com UV

O agente DEVE executar os seguintes comandos para inicializar o backend:

```
cd orientador-ia
uv init backend --python 3.12
cd backend
uv add fastapi uvicorn[standard] sqlalchemy[asyncio] asyncpg alembic \
    pydantic pydantic-settings python-jose[cryptography] passlib[bcrypt] \
    python-multipart httpx google-adk qdrant-client redis[hiredis] \
    python-dotenv aiofiles

# Dependências de desenvolvimento
uv add --dev pytest pytest-asyncio httpx
```

⚠ OBRIGATÓRIO: NUNCA criar requirements.txt. NUNCA usar 'pip install'. O Dockerfile DEVE usar 'uv run' para executar a aplicação.

4.2 Dockerfile do Backend

```
FROM python:3.12-slim

# Instalar UV
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

WORKDIR /app
COPY pyproject.toml uv.lock ./

# Instalar dependências via UV (sem criar venv, instala no sistema)
RUN uv sync --frozen --no-dev

COPY app/ ./app/

CMD ["uv", "run", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4.3 Schema SQL Completo (PostgreSQL)

O agente DEVE criar estes modelos SQLAlchemy em app/models/sql_models.py. Usar Alembic para as migrações.

Tabela: users

```
class User(Base):
    __tablename__ = 'users'
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid4)
    email = Column(String, unique=True, nullable=False, index=True)
    password_hash = Column(String, nullable=False)
    full_name = Column(String, nullable=False)
    academic_level = Column(Enum('HIGHSCHOOL', 'BACHELORS', 'MASTERS', 'PHD',
                                name='academic_level_enum'), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    projects = relationship('Project', back_populates='owner')
```

Tabela: projects

```
class Project(Base):
    __tablename__ = 'projects'
```

```

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey('users.id'),
nullable=False)
    title = Column(String, nullable=False)
    domain_area = Column(String, nullable=False)
    academic_level = Column(Enum('HIGHSCHOOL', 'BACHELORS', 'MASTERS', 'PHD',
                                name='academic_level_enum'), nullable=False)
    status = Column(Enum('DRAFT', 'REVIEW', 'ANALYSIS', 'COMPLETE',
                        name='project_status_enum'), default='DRAFT')
    human_guidelines = Column(Text, nullable=True) # System Prompt extra
    theoretical_alma_id = Column(UUID(as_uuid=True),
ForeignKey('ecosystem_resources.id'), nullable=True)
    methodological_alma_id = Column(UUID(as_uuid=True),
ForeignKey('ecosystem_resources.id'), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

```

Tabela: project_canvas_state

```

class ProjectCanvasState(Base):
    __tablename__ = 'project_canvas_state'
    project_id = Column(UUID(as_uuid=True), ForeignKey('projects.id'),
primary_key=True)
    canvas_json = Column(JSONB, nullable=False, default=lambda: {
        'tema': {'content': '', 'is_locked': False},
        'problema': {'content': '', 'is_locked': False},
        'justificativa': {'content': '', 'is_locked': False},
        'objetivos': {'geral': '', 'especificos': []},
        'metodologia': {'tipo': '', 'instrumentos': []}
    })
    updated_at = Column(DateTime, default=datetime.utcnow,
onupdate=datetime.utcnow)

```

Tabela: ecosystem_resources (Almas e Ferramentas)

```

class EcosystemResource(Base):
    __tablename__ = 'ecosystem_resources'
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid4)
    resource_type = Column(Enum('ALMA', 'SKILL', 'TOOL',
                                name='resource_type_enum'), nullable=False)
    name = Column(String, nullable=False)
    description = Column(Text, nullable=False) # Usado no match semântico
    alma_type = Column(Enum('THEORETICAL', 'METHODOLOGICAL',
                            name='alma_type_enum'), nullable=True)
    scope = Column(Enum('LOCAL', 'GLOBAL',
                        name='scope_enum'), default='GLOBAL')
    origin_project_id = Column(UUID(as_uuid=True), nullable=True)
    system_prompt = Column(Text, nullable=False) # Prompt base da Alma
    personality_descriptor = Column(String, nullable=False) # ex: 'Crítico,
Meticuloso, Socrático'
    is_approved = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)

```

Tabela: chat_messages

```

class ChatMessage(Base):
    __tablename__ = 'chat_messages'
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid4)

```



```
    project_id = Column(UUID(as_uuid=True), ForeignKey('projects.id'),
nullable=False, index=True)
    role        = Column(Enum('USER', 'ALMA', 'SYSTEM', name='role_enum'),
nullable=False)
    alma_id     = Column(UUID(as_uuid=True), nullable=True) # Qual Alma falou
    content     = Column(Text, nullable=False)
    created_at  = Column(DateTime, default=datetime.utcnow)
```

4.4 GraphState — Schema Canônico (IMUTÁVEL)

Este é o único schema de estado permitido. Todos os agentes ADK lêem e escrevem neste schema. Definir em `app/state/graph_state.py`.

```
from pydantic import BaseModel, Field
from typing import Optional, List, Any
from uuid import UUID

class CanvasState(BaseModel):
    tema: dict = Field(default={'content': '', 'is_locked': False})
    problema: dict = Field(default={'content': '', 'is_locked': False})
    justificativa: dict = Field(default={'content': '', 'is_locked': False})
    objetivos: dict = Field(default={'geral': '', 'especificos': []})
    metodologia: dict = Field(default={'tipo': '', 'instrumentos': []})

class ChatMessage(BaseModel):
    role: str # 'user' | 'alma' | 'system'
    alma_name: Optional[str] = None
    content: str
    timestamp: str

class ValidationFlags(BaseModel):
    is_plagiarism_attempt: bool = False
    needs_bibliography: bool = False
    plagiarism_confidence: float = 0.0 # 0.0 a 1.0

class GraphState(BaseModel):
    project_id: str
    user_id: str
    academic_level: str # 'HIGHSCHOOL' | 'BACHELORS' | 'MASTERS' | 'PHD'
    chat_history: List[ChatMessage] = Field(default_factory=list)
    current_canvas: CanvasState = Field(default_factory=CanvasState)
    active_theoretical_alma: str # nome da Alma Teórica ativa
    active_methodological_alma: str # nome do Avatar Metodológico ativo
    orchestrator_directive: str = '' # instrução interna do Maestro
    canvas_fields_to_update: dict = Field(default_factory=dict)
    validation_flags: ValidationFlags = Field(default_factory=ValidationFlags)
    human_guidelines: str = '' # System prompt extra do orientador humano
```

4.5 Google ADK — Implementação dos Agentes

O Google Agent Development Kit (ADK) é o único framework de orquestração autorizado. Instalar via: `uv add google-adk`

4.5.1 Agente Orquestrador ('O Maestro') — `app/agents/orchestrator.py`

O Maestro é o único ponto de entrada para todos os pedidos de chat. Decide qual Alma responde e aciona o Canvas Extractor.

```
import google.adk as adk
from app.state.graph_state import GraphState
from app.agents.guardrails import check_plagiarism
from app.agents.almas.base_alma import get_alma_by_name
from app.agents.canvas_extractor import extract_canvas_fields

ORCHESTRATOR_SYSTEM_PROMPT = """
Você é o Maestro, o orquestrador do Orientador.IA. O seu único papel é:
1. Receber a mensagem do utilizador e o GraphState completo.
2. Decidir qual das Almas ativas deve responder (SEMPRE a Alma Teórica PRIMEIRO,
   alternando com o Avatar Metodológico quando relevante para metodologia).
3. Detetar se a mensagem é um pedido de plágio (pedir para escrever o trabalho).
4. NUNCA gerar a resposta final – apenas emitir a directiva.

Responda SEMPRE com um JSON com esta estrutura exacta:
{
  "selected_alma": "THEORETICAL" | "METHODOLOGICAL",
  "is_plagiarism": true | false,
  "directive": "<instrução interna para a Alma>"
}
"""

orchestrator_agent = adk.Agent(
    name='maestro_orchestrator',
    model='ollama/llama3.1:8b',
    system_prompt=ORCHESTRATOR_SYSTEM_PROMPT,
    tools=[], # O Maestro NÃO usa ferramentas externas
    output_schema=OrchestratorOutput, # Pydantic model para JSON estruturado
)
```

4.5.2 Classe Base das Almas — `app/agents/almas/base_alma.py`

Todas as Almas herdam desta classe. O comportamento maiêutico é obrigatório.

```
import google.adk as adk
from app.services.ollama_client import OllamaClient

BASE_ALMA_INSTRUCTIONS = """
REGRAS ABSOLUTAS DE COMPORTAMENTO (nunca violar):
1. NUNCA escrever partes do trabalho académico do utilizador.
2. SEMPRE responder com perguntas socráticas que estimulem o pensamento crítico.
3. Manter SEMPRE o tom de voz e a perspectiva teórica definidos no teu System Prompt.
4. As tuas respostas têm um limite máximo de 250 palavras.
5. Quando detectares que o utilizador chegou a uma conclusão sobre Tema, Problema,
   Justificativa, Objectivo ou Metodologia, termina a resposta com o tag XML:
   <canvas_signal field="NOME_DO_CAMPO" value="TEXTO_CONCLUIDO" />
6. Adaptar a complexidade linguística ao nível académico do utilizador:
   HIGH SCHOOL → linguagem simples, exemplos concretos.
   PHD → terminologia técnica rigorosa, referências implícitas.

```

```

"""

class BaseAlma:
    def __init__(self, name: str, system_prompt: str, personality: str):
        self.name = name
        self.agent = adk.Agent(
            name=f'alma_{name.lower()}',
            model='ollama/llama3.1:8b',
            system_prompt=system_prompt + '\n\n' + BASE_ALMA_INSTRUCTIONS,
            tools=[],
        )

    async def stream_response(self, state: GraphState) -> AsyncIterator[str]:
        """Retorna um async generator com chunks de texto para SSE."""
        # Construir contexto completo para a Alma
        context = build_alma_context(state)
        async for chunk in self.agent.stream(context):
            yield chunk

```

4.5.3 As 10 Almas Estáticas do MVP

Cada ficheiro de Alma define apenas o `system_prompt` e o `personality_descriptor`. O comportamento base vem de `BaseAlma`.

Ficheiro	Nome	Tipo	Perspectiva Teórica (System Prompt Core)
foucault.py	Michel Foucault	THEORETICAL	Analisar todo o fenómeno como exercício de poder/saber. Usar conceitos: panóptico, biopoder, genealogia, discurso. Nunca afirmar — questionar as relações de poder implícitas.
bourdieu.py	Pierre Bourdieu	THEORETICAL	Analisar através de: habitus, campo, capital (económico, cultural, social, simbólico). Questionar as estruturas sociais reproduzidas de forma naturalizada.
freire.py	Paulo Freire	THEORETICAL	Pedagogia crítica, conscientização, educação bancária vs. libertadora. Relacionar SEMPRE com o contexto de emancipação e transformação social.
habermas.py	Jürgen Habermas	THEORETICAL	Acção comunicativa, esfera pública, racionalidade. Questionar a validade dos argumentos e as condições de comunicação ideal.
hooks.py	bell hooks	THEORETICAL	Interseccionalidade de raça, género e classe. Pedagogia engajada. Questionar quem é silenciado no discurso académico.
etnografa.py	A Etnógrafa Digital	METHODOLOGICAL	Especialista em etnografia digital, netnografia e observação participante online. Guiar o utilizador no design de instrumentos qualitativos para ambientes digitais.

Ficheiro	Nome	Tipo	Perspectiva Teórica (System Prompt Core)
estatistico.py	O Estatístico	METHODOLOGICAL	Especialista em métodos quantitativos, SPSS/R, escalas Likert, análise de regressão. Guiar o utilizador na operacionalização de variáveis e design experimental.
fenomenologa.py	A Fenomenóloga	METHODOLOGICAL	Especialista em fenomenologia husserliana e hermenêutica. Guiar o utilizador em entrevistas em profundidade e análise de experiências vividas.
grounded.py	O Grounded Theorist	METHODOLOGICAL	Especialista em Grounded Theory (Strauss & Corbin). Codificação aberta/axial/selectiva. Guiar o utilizador na construção de teoria a partir dos dados.
narrativo.py	O Analista Narrativo	METHODOLOGICAL	Especialista em análise narrativa e análise de conteúdo. Guiar o utilizador na identificação de categorias temáticas e estruturas narrativas.

4.5.4 Canvas Extractor Agent — app/agents/canvas_extractor.py

Este agente corre em background após cada turno de chat. Analisa o histórico de conversa e extrai campos preenchidos do Canvas.

```
CANVAS_EXTRACTOR_PROMPT = """
Analisar o histórico de chat fornecido e identificar se o utilizador expressou
conclusões
claras sobre algum dos seguintes campos do Canvas de Investigação:
- tema: O tema central do trabalho
- problema: A questão/problema de investigação central
- justificativa: A relevância/justificação do estudo
- objetivos_geral: O objectivo geral do estudo
- objetivos_especificos: Lista de objectivos específicos
- metodologia_tipo: Tipo de metodologia (qualitativa/quantitativa/mista)
- metodologia_instrumentos: Lista de instrumentos/técnicas de recolha de dados

Extrair APENAS o que foi explicitamente afirmado pelo utilizador (role='user').
Ignorar perguntas e reflexões. Só extrair certezas.

Responder EXCLUSIVAMENTE com JSON:
{
  "fields_updated": {
    "tema": "<texto ou null>",
    "problema": "<texto ou null>",
    "justificativa": "<texto ou null>",
    "objetivos_geral": "<texto ou null>",
    "objetivos_especificos": ["<item1>", ...] ou null,
    "metodologia_tipo": "<texto ou null>",
    "metodologia_instrumentos": ["<item1>", ...] ou null
  }
}
"""

canvas_extractor_agent = adk.Agent(
    name='canvas_extractor',
    model='ollama/llama3.1:8b',
    system_prompt=CANVAS_EXTRACTOR_PROMPT,
    tools=[],
    output_schema=CanvasExtractionOutput,
)
```

4.5.5 Guardrails Agent — app/agents/guardrails.py

Usa o modelo Mistral 7B local (mais rápido). Corre ANTES do Orchestrator em cada mensagem.

```
GUARDRAIL_PROMPT = """
Analisar a mensagem do utilizador e classificar se é uma tentativa de obter
conteúdo que viole a integridade académica. Exemplos de violações:
- 'Escreve a minha introdução'
- 'Faz o resumo do meu artigo'
- 'Redige a conclusão'
- 'Gera o meu abstract'

NÃO é violação:
- Pedir ajuda para estruturar o raciocínio
- Pedir exemplos de como formular uma questão
- Pedir feedback sobre uma ideia

```

```
Responder APENAS com JSON: {"is_violation": true | false, "confidence": 0.0-1.0}
"""
```

```
guardrail_agent = adk.Agent(
    name='guardrail_checker',
    model='ollama/mistral:7b', # Modelo local dedicado para Guardrails
    system_prompt=GUARDRAIL_PROMPT,
    tools=[],
    output_schema=GuardrailOutput,
)
```

```
PLAGIARISM_RESPONSE = """
```

```
Entendo o teu desafio com a página em branco! No entanto, o Orientador.IA não redige
partes do teu trabalho – isso comprometeria o teu próprio aprendizado e a integridade
académica do projeto. O que posso fazer é ajudar-te a PENSAR o que queres escrever.
Tenta responder: qual é a ideia central que queres transmitir nesta secção?
"""
```

4.6 API Endpoints — Especificação Completa

Autenticação — /api/auth/

Método	Endpoint	Request Body	Response	Notas
POST	/api/auth/register	{email, password, full_name, academic_level}	{access_token, token_type}	Hash da password com bcrypt. Retornar JWT.
POST	/api/auth/login	{email, password}	{access_token, token_type}	Verificar hash. JWT expira em ACCESS_TOKEN_EXPIRE_MINUTES.
GET	/api/auth/me	— (JWT header)	{id, email, full_name, academic_level}	Requer Authorization: Bearer <token>

Projetos — /api/projects/

Método	Endpoint	Request Body	Response
GET	/api/projects/	—	List[ProjectOut] (todos os projetos do user autenticado)
POST	/api/projects/	{title, domain_area, academic_level, human_guidelines?}	ProjectOut (com canvas_state vazio)
GET	/api/projects/{id}	—	ProjectOut + canvas_state + chat_history (últimas 50 msgs)
PATCH	/api/projects/{id}	{title?, domain_area?, status?, human_guidelines?}	ProjectOut atualizado
DELETE	/api/projects/{id}	—	{success: true}
POST	/api/projects/{id}/match	{raw_idea: string}	MatchResult (ver 4.7)
POST	/api/projects/{id}/select-almas	{theoretical_alma_id, methodological_alma_id}	ProjectOut atualizado
GET	/api/projects/{id}/canvas	—	CanvasState JSON
PATCH	/api/projects/{id}/canvas	{field: string, value: any}	CanvasState JSON atualizado

Chat — /api/chat/

Método	Endpoint	Protocolo	Descrição
GET/WS	/api/chat/{project_id}/ws	WebSocket	Conexão persistente. Client envia: {type:'message', content:'...'} e Server emite: {type:'chunk', text:'...'} e {type:'canvas_update', canvas:{...}}
GET	/api/chat/{project_id}/history	HTTP GET	Retorna List[ChatMessageOut] (todos os registos da tabela chat_messages para o projecto)

4.7 Match Engine — Implementação Detalhada

O Match Engine usa similaridade semântica via Qdrant para sugerir Almas. Implementar em `app/agents/match_engine.py`.

- PASSO 1: Receber `raw_idea` (texto livre do utilizador).
- PASSO 2: Gerar embedding da `raw_idea` via `OllamaClient` com modelo `nomic-embed-text`.
- PASSO 3: Pesquisar no Qdrant na collection `almas_catalog` com o embedding.
- PASSO 4: Filtrar por `payload.alma_type = 'THEORETICAL'` → retornar top 3.
- PASSO 5: Filtrar por `payload.alma_type = 'METHODOLOGICAL'` → retornar top 3.
- PASSO 6: Retornar `MatchResult` com os 6 resultados ordenados por score de similaridade.

NOTA: A collection `almas_catalog` no Qdrant deve ser populada pelo script `scripts/seed_almas.py` ao iniciar o sistema. Cada Alma tem um vector gerado a partir do seu `description` field.

4.8 Ollama Client — `app/services/ollama_client.py`

Wrapper que abstrai todas as chamadas ao servidor Ollama local.

```
import httpx
from app.config import settings

class OllamaClient:
    def __init__(self):
        self.base_url = settings.OLLAMA_BASE_URL
        self.client = httpx.AsyncClient(timeout=120.0)

    async def chat_stream(self, model: str, messages: list, system: str = None):
        """Streaming: async generator de chunks de texto."""
        payload = {'model': model, 'messages': messages, 'stream': True}
        if system:
            payload['system'] = system
        async with self.client.stream('POST', f'{self.base_url}/api/chat',
                                     json=payload) as resp:
            async for line in resp.aiter_lines():
                if line:
                    data = json.loads(line)
                    if not data.get('done'):
                        yield data['message']['content']

    async def embed(self, text: str) -> list[float]:
        """Gerar embedding para um texto."""
        resp = await self.client.post(
            f'{self.base_url}/api/embeddings',
            json={'model': settings.OLLAMA_EMBED_MODEL, 'prompt': text}
        )
        return resp.json()['embedding']
```

5. FRONTEND — ESPECIFICAÇÃO DETALHADA

5.1 Inicialização do Projecto

```
cd orientador-ia
npx create-next-app@latest frontend \
  --typescript --tailwind --eslint --app --src-dir --import-alias '@/*'
cd frontend
npx shadcn@latest init
npm install zustand socket.io-client

# Componentes shadcn obrigatórios para o MVP
npx shadcn@latest add button input textarea card badge separator toast
```

5.2 Rotas da Aplicação (App Router)

Rota	Ficheiro	Descrição
/	src/app/page.tsx	Redirect para /login se não autenticado, /dashboard se autenticado
/login	src/app/(auth)/login/page.tsx	Formulário de login com email/password
/register	src/app/(auth)/register/page.tsx	Formulário de registo com email, nome, password, nível académico
/dashboard	src/app/(dashboard)/page.tsx	Lista de projectos do utilizador. Botão 'Novo Projeto'.
/project/[id]	src/app/project/[id]/page.tsx	Página principal do Ateliê: Chat (esquerda) + Canvas (direita)
/project/[id]/match	src/app/project/[id]/match/page.tsx	Wizard de match: input de ideia → sugestão de Almas → confirmação

5.3 Zustand Store — src/store/project.ts

Estado global do projecto activo. Inclui Canvas e histórico de chat.

```
import { create } from 'zustand'

interface ProjectStore {
  canvas: CanvasState
  chatMessages: ChatMessage[]
  isStreaming: boolean
  activeAlmas: { theoretical: AlmaInfo | null, methodological: AlmaInfo | null }
  updateCanvas: (field: string, value: any) => void
  addMessage: (msg: ChatMessage) => void
  appendToLastMessage: (chunk: string) => void
  setStreaming: (v: boolean) => void
}

export const useProjectStore = create<ProjectStore>((set) => ({
  canvas: defaultCanvas,
  chatMessages: [],
  isStreaming: false,
  activeAlmas: { theoretical: null, methodological: null },
  updateCanvas: (field, value) => set((s) => ({
    canvas: { ...s.canvas, [field]: value }
  }
  )
```

```

    })),
    addMessage: (msg) => set((s) => ({
      chatMessages: [...s.chatMessages, msg]
    })),
    appendToLastMessage: (chunk) => set((s) => {
      const msgs = [...s.chatMessages]
      if (msgs.length > 0) {
        msgs[msgs.length - 1].content += chunk
      }
      return { chatMessages: msgs }
    })),
    setStreaming: (v) => set({ isStreaming: v }),
  })
)

```

5.4 WebSocket Client — src/lib/ws.ts

```

export class ChatSocket {
  private ws: WebSocket | null = null

  connect(projectId: string, token: string,
    onChunk: (text: string) => void,
    onCanvasUpdate: (canvas: CanvasState) => void) {

    const url =
`${process.env.NEXT_PUBLIC_WS_URL}/api/chat/${projectId}/ws?token=${token}`
    this.ws = new WebSocket(url)

    this.ws.onmessage = (event) => {
      const data = JSON.parse(event.data)
      if (data.type === 'chunk') onChunk(data.text)
      if (data.type === 'canvas_update') onCanvasUpdate(data.canvas)
      if (data.type === 'done') { /* stream concluído */ }
      if (data.type === 'guardrail_block') { /* mostrar mensagem de integridade */ }
    }
  }

  sendMessage(content: string) {
    this.ws?.send(JSON.stringify({ type: 'message', content }))
  }

  disconnect() { this.ws?.close() }
}

```

5.5 Componente CanvasPanel — src/components/canvas/CanvasPanel.tsx

Painel lateral que exibe e permite editar os campos do Canvas em tempo real.

- Exibir os 7 campos do Canvas: Tema, Problema, Justificativa, Objectivo Geral, Objectivos Específicos, Tipo de Metodologia, Instrumentos.
- Campos preenchidos pelo sistema aparecem com fundo levemente colorido e badge 'Auto'.
- Clicar num campo permite edição manual (textarea). Ao confirmar, emitir PATCH `/api/projects/{id}/canvas`.
- Campos com `is_locked=true` mostram um ícone de cadeado e não são editáveis.
- Botão 'Exportar Canvas (JSON)' no fundo do painel — descarrega o `canvas_json` atual.

6. DOCKER COMPOSE — CONFIGURAÇÃO COMPLETA

Criar na raiz do projecto. Este ficheiro orquestra todos os serviços do MVP.

```
version: '3.9'

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: orientador_db
      POSTGRES_USER: orientador
      POSTGRES_PASSWORD: orientador_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U orientador -d orientador_db']
      interval: 5s
      retries: 5

  qdrant:
    image: qdrant/qdrant:v1.9.0
    volumes:
      - qdrant_data:/qdrant/storage
    ports:
      - '6333:6333'
      - '6334:6334'

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data

  ollama:
    image: ollama/ollama:latest
    volumes:
      - ollama_data:/root/.ollama
    ports:
      - '11434:11434'
    # Para usar GPU (opcional):
    # deploy:
    #   resources:
    #     reservations:
    #       devices:
    #         - driver: nvidia
    #           capabilities: [gpu]

  backend:
    build: ./backend
    ports:
      - '8000:8000'
    env_file: .env
    depends_on:
      postgres:
        condition: service_healthy
```

```
  qdrant:
    condition: service_started
  redis:
    condition: service_started
  ollama:
    condition: service_started
volumes:
  - ./backend/app:/app/app # Hot reload em desenvolvimento

frontend:
  build: ./frontend
  ports:
    - '3000:3000'
  env_file: .env
  depends_on:
    - backend

volumes:
  postgres_data:
  qdrant_data:
  redis_data:
  ollama_data:
```

7. MODELOS OLLAMA — SETUP OBRIGATÓRIO

Após 'docker compose up -d ollama', executar os seguintes comandos para descarregar os modelos necessários:

```
# Modelo principal de chat (Almas e Orchestrator)
docker compose exec ollama ollama pull llama3.1:8b

# Modelo para Guardrails (mais rápido)
docker compose exec ollama ollama pull mistral:7b

# Modelo de embeddings (obrigatório para Match Engine e RAG)
docker compose exec ollama ollama pull nomic-embed-text
```

Modelo Ollama	Uso	Dimensões (embed)	Tamanho Aprox.
llama3.1:8b	Chat (Almas), Orchestrator, Canvas Extractor	—	~4.7 GB
mistral:7b	Guardrails (classificação binária rápida)	—	~4.1 GB
nomic-embed-text	Embeddings (Match Engine, RAG Fase 2)	768	~274 MB

⚠ OBRIGATÓRIO: O servidor Ollama DEVE estar operacional e os modelos descarregados ANTES do backend iniciar. O backend deve verificar a saúde do Ollama no startup e falhar com mensagem de erro clara se os modelos não estiverem disponíveis.

8. SCRIPTS DE INICIALIZAÇÃO OBRIGATÓRIOS

8.1 Script de Seed das Almas — scripts/seed_almas.py

Este script DEVE ser executado uma vez após o primeiro 'docker compose up'. Popula a base de dados SQL e o Qdrant com as 10 Almas do MVP.

```
#!/usr/bin/env python
"""
Seed script para popular ecosystem_resources (SQL) e almas_catalog (Qdrant)
Executar: uv run python scripts/seed_almas.py
"""

ALMAS_DATA = [
    {
        'name': 'Michel Foucault',
        'alma_type': 'THEORETICAL',
        'personality_descriptor': 'Crítico, Genealógico, Desconstrutivo',
        'description': 'Teórico das relações de poder, vigilância, discurso e biopoder. '
        'Ideal para investigações sobre controlo social, instituições, '
        'tecnologias de poder e subjectividade.',
        'system_prompt': '(ver Secção 4.5.3)'
    },
    # ... (repetir para todas as 10 Almas)
]

async def seed():
    # 1. Inserir em PostgreSQL via SQLAlchemy
    # 2. Criar collection 'almas_catalog' no Qdrant com dimensão 768
    # 3. Para cada Alma: gerar embedding do 'description' via Ollama nomic-embed-text
    # 4. Inserir no Qdrant com payload: {id, name, alma_type, personality_descriptor}
    pass
```

8.2 Migrações Alembic

```
cd backend
# Inicializar Alembic (apenas uma vez)
uv run alembic init alembic

# Gerar migração a partir dos modelos
uv run alembic revision --autogenerate -m 'initial_schema'

# Aplicar migrações
uv run alembic upgrade head
```

8.3 Sequência de Arranque Completa

O agente DEVE executar os comandos nesta ordem exacta:

- 1. docker compose up -d postgres qdrant redis ollama
- 2. Aguardar 10 segundos
- 3. docker compose exec ollama ollama pull llama3.1:8b
- 4. docker compose exec ollama ollama pull mistral:7b
- 5. docker compose exec ollama ollama pull nomic-embed-text

- 6. `cd backend && uv run alembic upgrade head`
- 7. `uv run python ../scripts/seed_almas.py`
- 8. `docker compose up -d backend frontend`
- 9. Verificar: <http://localhost:3000> (frontend) e <http://localhost:8000/docs> (API Swagger)

9. CRITÉRIOS DE ACEITAÇÃO DO MVP

O MVP está completo quando TODOS os seguintes critérios estiverem satisfeitos:

9.1 Funcionalidades Obrigatórias

ID	Critério	Como Testar
AC-01	Utilizador consegue registar-se com email/password e fazer login	POST /api/auth/register → receber JWT
AC-02	Utilizador consegue criar um projecto com título, área e nível académico	POST /api/projects/ → projecto criado no PostgreSQL
AC-03	Sistema sugere 3 Almas Teóricas e 3 Avatares Metodológicos relevantes para a ideia do utilizador	POST /api/projects/{id}/match com texto de ideia → retorna 6 sugestões ordenadas por score
AC-04	Utilizador consegue seleccionar 1 Alma Teórica e 1 Avatar Metodológico para o projecto	POST /api/projects/{id}/select-almas → project.theoretical_alma_id e methodological_alma_id actualizados
AC-05	Mensagem de chat chega ao utilizador via streaming (SSE/WebSocket)	WebSocket: enviar mensagem → receber chunks de texto em tempo real (<3s para primeiro chunk)
AC-06	A Alma nunca escreve partes do trabalho académico	Enviar 'Escreve a minha introdução' → receber PLAGIARISM_RESPONSE (não conteúdo)
AC-07	O Canvas é actualizado automaticamente quando o sistema detecta um campo definido	Após diálogo onde utilizador conclui o Problema → campo 'problema' no Canvas actualiza automaticamente no Frontend
AC-08	Utilizador consegue editar manualmente um campo do Canvas	PATCH /api/projects/{id}/canvas → campo actualizado na DB e emitido via WebSocket
AC-09	Histórico de chat persiste entre sessões	GET /api/chat/{id}/history → retorna mensagens anteriores
AC-10	Exportação do Canvas como JSON funciona	Botão 'Exportar Canvas' → descarregar ficheiro JSON com todos os campos

9.2 Requisitos de Performance

- Tempo de resposta do primeiro chunk de chat: < 3 segundos (hardware: CPU moderna, 16GB RAM)
- Tempo de resposta do Match Engine: < 2 segundos para 10 Almas em Qdrant
- Tempo de extracção do Canvas (background): < 5 segundos após o turno de chat
- Todos os endpoints da API devem responder em < 500ms (excluindo geração de LLM)

9.3 Qualidade de Código

- Todo o código Python DEVE passar: uv run ruff check app/ sem erros
- Todo o código TypeScript DEVE passar: npm run build sem erros
- O docker-compose up deve resultar em todos os serviços healthy em menos de 2 minutos

10. FORA DO SCOPE DO MVP — NÃO IMPLEMENTAR

Para evitar scope creep, o agente DEVE recusar implementar qualquer um dos seguintes itens nesta iteração:

Funcionalidade	Razão da Exclusão	Fase Futura
Login OAuth (Google/Institucional)	Complexidade desnecessária para validar o conceito	Fase 2
Pipeline RAG completo (Agente Buscador)	Requer infraestrutura adicional (Playwright, APIs externas)	Fase 2
Exportação de Canvas em PDF/DOCX	Biblioteca de geração de documentos fora do scope MVP	Fase 2
Mesa-Redonda (análise de dados empíricos)	Requer processamento assíncrono heavy (Celery, OCR)	Fase 3
Upload de ficheiros (PDF, transcrições)	Dependente da Mesa-Redonda (Fase 3)	Fase 3
Agente Ferramenteiro (geração de scripts)	Requer Sandboxing seguro (Docker-in-Docker, gVisor)	Fase 4
Agente Génesis (criação dinâmica de Almas)	Dependente de validação e pipeline evolutivo	Fase 4
Motor Evolutivo (LOCAL→GLOBAL)	Requer pipeline de validação e AST parsing	Fase 4
Guardrails RAG (anti-alucinação bibliográfica)	Depende do Vector Store populado (Fase 2)	Fase 2
Rate Limiting por utilizador/token	Não relevante para MVP com modelos locais Ollama	Fase 2

FIM DA ESPECIFICAÇÃO TÉCNICA DO MVP — ORIENTADOR.IA v1.0

Qualquer desvio desta especificação deve ser tratado como um bug, não como uma feature.